СЕМИНАР ПО ЛЕКЦИИ 4

Планы, индексы и схема хранилища

В начале — краткая теория по лекции. Затем задания: в первой части разбираем планы запросов и индексы, во второй вместе проектируем хранилище для нашего магазина в Greenplum. Все запросы заданий есть в файле `seminar_part1.sql`.

КРАТКАЯ ТЕОРИЯ ПО ЛЕКЦИИ

1 Как база выполняет запрос

Мы пишем в SQL, **что** хотим получить, а **как** это сделать, решает база. Запрос проходит пять стадий. Каждую удобно запомнить по ошибке, которую она ловит:

Стадия	Что проверяет	Пример ошибки
Парсер	написан ли запрос по правилам SQL	`syntax error at or near "selec"`
Анализ	есть ли такие таблицы и столбцы, совпадают ли типы	`relation "order_item" does not exist`
Rewrite	подставляет вместо представлений (VIEW) их запросы	—
Планировщик	выбирает самый быстрый способ выполнения	ошибки нет, но план может быть медленным
Исполнитель	выполняет план и отдаёт результат	`division by zero`

Планировщик выбирает способ по **статистике** — сводке о данных: сколько строк в таблице и какие значения встречаются чаще. Статистику собирает команда `analyze`.

2 Как читать EXPLAIN

- `explain` — только показывает план, запрос не выполняется.
- `explain analyze` — выполняет запрос и показывает фактические цифры. Осторожно с `update` и `delete`: изменения применятся, поэтому оборачиваем их в `begin; ... rollback;`.
- `explain (analyze, buffers)` — плюс сколько страниц данных прочитано. Удобнее всего использовать именно его.

```
Limit  (cost=100822.91..100822.96 rows=20) (actual time=143.347..143.349 rows=20 loops=1)
  ->  Sort  (cost=100822.91..100823.39 rows=193) (actual time=124.181..124.182 rows=20
loops=1)
        ->  Seq Scan on orders_big  (cost=0.00..100817.77 rows=193)
                                    (actual time=0.461..124.133 rows=108 loops=1)
              Filter: (customer_id = 4217)
              Rows Removed by Filter: 5075892
              Buffers: shared read=37376
Execution Time: 163.157 ms
```

План читаем снизу вверх: самый вложенный шаг выполняется первым. Здесь база прочитала всю таблицу (Seq Scan), отсортировала найденное (Sort) и взяла 20 строк (Limit).

Что в плане	Что значит
`cost=...`	оценка в условных единицах, **не** миллисекунды
`rows` в первой скобке	сколько строк база **ожидала**
`rows` во второй скобке	сколько строк получилось **на самом деле**
`loops`	сколько раз выполнялся шаг. Если больше 1 — строки и время умножаем на `loops`
`Rows Removed by Filter`	сколько строк прочитали и выбросили
`Buffers`	сколько страниц по 8 КБ прочитано: `hit` — из памяти, `read` — с диска
`Execution Time`	общее время запроса

На что смотреть в первую очередь: где время резко растёт; сильно ли отличается прогноз строк от факта; нет ли большого `Rows Removed by Filter`; не ушла ли сортировка на диск (`external merge Disk`).

3 Почему план бывает плохим

Планировщик выбирает способ под то число строк, которое **ожидает**. Если он ждёт 253 строки, а приходит 333 700, план будет плохим. Частые причины:

- **Статистика устарела** — например, в таблицу загрузили новые данные и не обновили статистику. Решение: после большой загрузки выполнить `analyze имя_таблицы;`.
- **Функция над столбцом** — про `lower(status)` или `date(order_created_at)` статистики нет, и база оценивает число строк наугад.

4 Индексы

Индекс — как предметный указатель в книге: значения столбца лежат в нём по порядку, а рядом с каждым — адрес строки в таблице.

```
create index idx_orders_big_customer_created
    on orders_big (customer_id, order_created_at desc);
-- запрос «последние 20 заказов клиента»: было 163 мс, стало 0,1 мс
```

- **Индекс помогает, когда нужно найти мало строк.** Если под условие подходит большая часть таблицы, прочитать её подряд быстрее, чем ходить через индекс. На маленькой таблице индекс тоже не нужен.
- **Составной индекс** строится по нескольким столбцам и работает как телефонный справочник: сначала по фамилии, внутри — по имени. Найти всех Ивановых легко, а всех Петров без фамилии — нет. Это **правило левого префикса**: первым ставим столбец, который сравниваем на равенство (`=`), за ним — столбец для диапазона или сортировки.
- **Типы индексов:** B-tree — обычный, подходит почти всегда; GIN — поиск внутри значения (слова в тексте, элементы массива, jsonb); BRIN — для огромных таблиц, куда данные пишутся по времени.
- **Индекс не бесплатный:** занимает место и замедляет вставку и изменение строк. Создаём его под конкретный медленный запрос и сравниваем план до и после.

5 Типовые ошибки в запросах

Ошибка	Плохо	Хорошо
Функция над столбцом	`where date(created_at) = '2026-06-15'`	`where created_at >= '2026-06-15'` `and created_at < '2026-06-16'`
Размножение строк соединением	сумма заказов после `join` с платежами: 5 200 вместо 3 100	сначала свести платежи к одной строке на заказ, потом соединять
Лишние столбцы	`select *`	только нужные столбцы
Сравнение с другим типом	`where session_id = 1`	`where session_id = 's-1'`
Далёкий offset	`offset 1000000 limit 20` — читает 1 000 020 строк	`where id > :последний_id order by id limit 20`

Порядок работы с медленным запросом: снять план → найти самый долгий шаг и сравнить прогноз строк с фактом → исправить статистику, потом текст запроса, потом добавить индекс → снять план ещё раз.

6 Факты и измерения

Для отчётов данные перекладывают в отдельное **хранилище** и делят таблицы на два вида.

- **Факт** — событие и числа, которые можно посчитать: позиция заказа (2 шт., цена 100, скидка 100), платёж, клик на сайте. Таблицы фактов узкие и очень длинные.
- **Измерение** — описание, по которому группируем и фильтруем: товар, клиент, дата. Таблицы измерений широкие и короткие.

Зерно — что означает одна строка таблицы фактов: «один товар в одном заказе», «один платёж». Его выбирают первым. Главная ошибка — смешать разное зерно в одной таблице: если положить сумму заказа рядом с платежами, заказ с двумя платежами посчитается дважды.

Вид метрики	Можно складывать?	Пример

Аддитивная	да, по любому разрезу	количество, сумма, скидка
Полуаддитивная	да, но не по времени	остаток на складе
Неаддитивная	нет	цена, процент, конверсия — считаем из сумм и количеств

У строки измерения два ключа: **натуральный** — номер из исходной системы (`customer_id = 1`), и **суррогатный** — собственный номер в хранилище (`customer_sk`). Отдельно строят **измерение дат** `dim_date`: одна строка — один день, год, месяц и выходной уже посчитаны.

7 Снежинка и звезда

- **Снежинка** — описание разбито на цепочку таблиц: товар → категория → родительская категория. Наша база `shop` устроена так, и для отчёта по верхним категориям нужно соединить шесть таблиц.
- **Звезда** — в центре таблица фактов, вокруг измерения, и каждое измерение — одна таблица со всем описанием. В `dim_product` сразу лежат товар, бренд, категория и все её родители. Запросы получаются короткими и однотипными.

Обычно выбирают звезду: измерения маленькие, поэтому повторы в них почти не занимают места, а запросы проще и быстрее.

8 Если данные в измерении меняются (SCD)

- **Type 1 — перезаписать.** Просто, но история теряется: если клиент переехал, все его прошлые заказы «переедут» вместе с ним.
- **Type 2 — хранить версии.** Старую строку закрываем датой окончания, добавляем новую версию со своим суррогатным ключом. Каждый заказ ссылается на версию, действовавшую в день заказа.

```
customer_sk | customer_id | city   | valid_from | valid_to   | is_current
          1 |           1 | Moscow | 2026-02-15 | 2026-06-15 | false
         41 |           1 | Kazan  | 2026-06-15 | 9999-12-31 | true
```

Решение принимают для каждого атрибута отдельно: опечатку в названии исправляют перезаписью, а переезд хранят версиями.

ЗАДАНИЯ

1 Угадай план

Для каждого запроса сначала скажите, как, по-вашему, база его выполнит: прочитает всю таблицу или пойдёт по индексу. Потом выполните запрос и посмотрите на самый нижний шаг плана, число строк и время.

В `orders_big` около 5 млн строк. Индексы есть по `order_id` (первичный ключ) и по `status`.

1.1 Маленькая таблица, поиск по первичному ключу

```
explain analyze
select * from orders where order_id = 113;
```

1.2 Большая таблица, поиск по первичному ключу

```
explain analyze
select * from orders_big where order_id = 2000113;
```

1.3 Оплаченные заказы

```
explain analyze
select count(*) from orders_big where status = 'paid';
```

1.4 Возвращённые заказы

```
explain analyze
select order_id, total_amount from orders_big where status = 'returned';
```

Вопросы для обсуждения:

- Почему в 1.1 база не использует первичный ключ, а в 1.2 использует?
- В 1.3 и 1.4 один и тот же индекс по `status`. Почему база берёт его только в одном случае?

2 Почини запрос

Поддержке нужны последние 20 заказов клиента 4217. Запрос работает, но медленно. Ускорьте его индексом.

2.1 Выполните запрос

```
explain (analyze, buffers)
select order_id, order_created_at, total_amount
from orders_big
where customer_id = 4217
order by order_created_at desc
limit 20;
```

Посмотрите на нижний шаг плана, `Rows Removed by Filter`, `Buffers` и `Execution Time`.

2.2 Создайте индекс

Индекс должен находить заказы клиента сразу отсортированными по дате. Какие столбцы и в каком порядке?

```
create index idx_orders_big_customer_created
    on orders_big (/* TODO */);
```

2.3 Выполните запрос из шага 2.1 ещё раз

Сравните план, `Buffers` и время с первым запуском.

Вопросы для обсуждения:

- Сколько строк база прочитала и выбросила в шаге 2.1, чтобы найти 20 заказов?
- Почему после индекса из плана исчезла сортировка?
- Почему в индексе первым стоит `customer_id`, а не дата?

3 Проектируем хранилище для магазина

Строим модель хранилища для нашего магазина `shop` всей группой. Идём по шагам и на каждом принимаем решение.

Шаг 1. Вопросы бизнеса

На какие вопросы должно отвечать хранилище? Например: какая выручка по категориям товаров по месяцам? Предложите свои вопросы и выберите 4–5 главных.

Шаг 2. Процессы

Какие события в магазине стоят за этими вопросами? Какие таблицы `shop` о них хранят данные?

Шаг 3. Зерно

Для каждой таблицы фактов закончите фразу «одна строка — это…».

- Можно ли положить сумму заказа и платежи в одну таблицу фактов?
- Все ли заказы из `shop` нужны в отчётах?
- Что делать с повторяющимися строками в `raw_events`?

Шаг 4. Измерения

По чему мы будем группировать и фильтровать каждый факт? Составьте таблицу: строки — таблицы фактов, столбцы — измерения (дата, клиент, товар, …). Какие измерения нужны всем фактам?

- Что делать, если у события на сайте нет клиента?
- У платежа есть дата заказа и дата оплаты. Сколько нужно таблиц дат?

Шаг 5. Атрибуты и изменения

Какие атрибуты будут у клиента и у товара? Для каждого решите: при изменении перезаписываем (Type 1) или храним версии (Type 2)?

- Клиент переехал в другой город. Какой тип выбрать и почему?
- Товар перевели в другую категорию. Должна ли прошлая выручка переехать вместе с ним?

Шаг 6. Метрики

Какие числа будут в таблицах фактов? Какие из них можно складывать, а какие — нет? Как правильно посчитать среднюю цену и конверсию?

Шаг 7. Проверка запросом

Возьмите один вопрос из шага 1 и напишите SQL к получившейся схеме. Если запрос не пишется или получается слишком длинным — в схеме чего-то не хватает.

Шаг 8. Как хранить таблицы в Greenplum

Хранилище нашего проекта будет в Greenplum. В первой части мы ускоряли запрос индексом в PostgreSQL. В Greenplum на скорость отчётов сильнее влияют три других решения:

- **Распределение.** Greenplum хранит каждую таблицу частями на нескольких серверах — сегментах — и считает на них одновременно. На каком сегменте лежит строка, решает ключ распределения: `distributed by (столбец)`. Хороший ключ раскладывает строки поровну. Если две большие таблицы часто соединяют по одному столбцу и обе распределены по нему, соединение идёт на каждом сегменте без пересылки строк по сети.
- **Копия на всех сегментах.** Маленькую таблицу можно целиком скопировать на каждый сегмент: `distributed replicated`. Тогда соединение с ней тоже не требует пересылки.
- **Колоночное хранение и партиции.** Колоночная таблица читает с диска только нужные запросу столбцы. Партиции по дате позволяют запросу за месяц читать только этот месяц.

- По какому столбцу распределить факты про заказы: позиции, платежи и заказы целиком? Что это даст при их соединении?
- Как хранить измерения — распределить по ключу или скопировать на все сегменты?
- Какие таблицы хранить по столбцам, а какие — обычно? Подсказка: что происходит с таблицей после загрузки — её только дописывают или ещё и обновляют?
- По какому столбцу делить таблицы фактов на партиции?

4 Результат семинара

После семинара оформите **схему и описание таблиц** хранилища по файлу `Шаблон описания хранилища.md`:

- схема со всеми таблицами фактов и измерений, их столбцами и связями;
- вопросы бизнеса, на которые отвечает хранилище;
- для каждой таблицы — назначение, что означает одна строка, источник в `shop`, столбцы с типами и описанием;
- для каждой таблицы — как она хранится в Greenplum и почему;
- запросы, которыми проверяется правильность загрузки.